

C++

Programming:

An Object-Oriented Approach

Chapter 11

Relationship Between Classes

C++

Programming:

An Object-Oriented
Approach



BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

BEHROUZ A. FOROUZAN
RICHARD F. GILBERG

Objectives Part 1

To define inheritance relationships between classes including public, protected, and private inheritance.

To learn how to write class definitions and member function definitions for several classes in an inheritance relationship.

To define association as a relationship between classes and its variations: aggregation and composition.

To learn how to write class definitions and member function definitions for several classes in an association.

Objectives Part 2

To define the dependency relationship between classes to show that one class can use the objects of another class.

To learn how to write class definitions and member function definitions for classes in a dependency relationship.

INHERITANCE Part 1

Inheritance in object-oriented programming derives a more specific concept from a more general one.

That is what we do in real life.

For example, the concept of *animal* in biology is more general than the concept of *horse*.

INHERITANCE Part 2

We can define the definition of an animal, we can then add to the definition to create the definition of a horse.

There is an "*is-a*" relation from the more specific to the more general.

All horses are animals, but not all animals are horses.

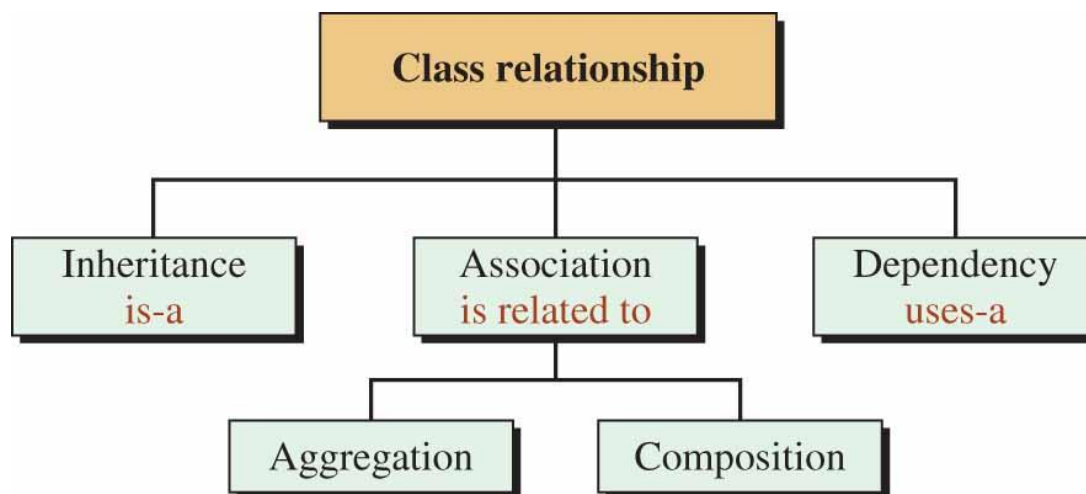
General Idea Part 1

In object-oriented programming, classes are not used in isolation.

They are used in relation to each other.

A program normally uses several classes with different relationships between them.

Figure 11.1 *Relationship between classes*



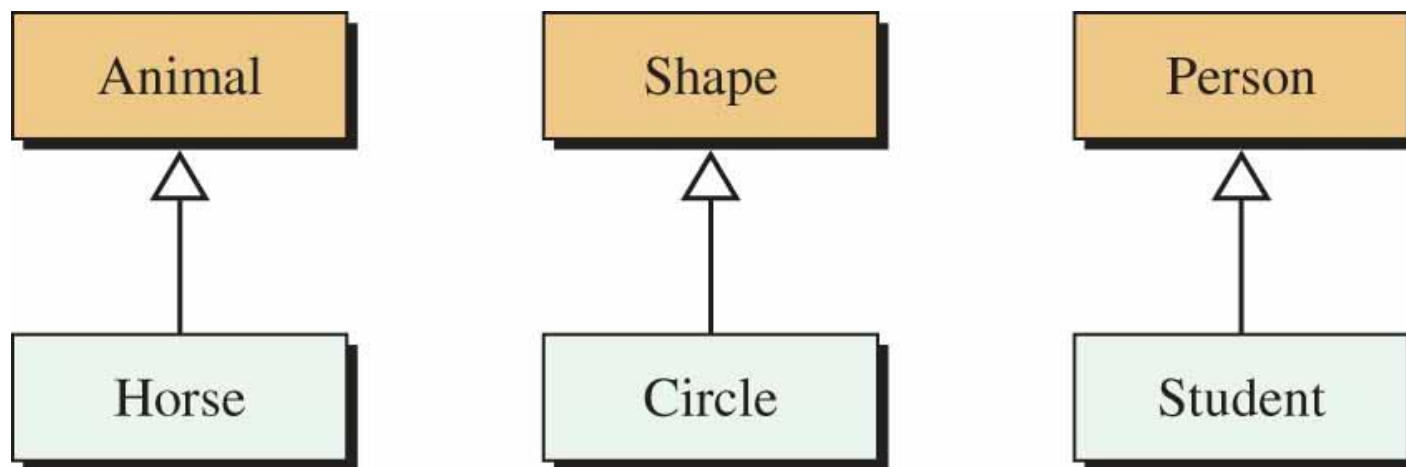
General Idea Part 2

To show the relationship between classes in inheritance, we use the Unified Modeling Language (UML).

The classes are shown as rectangle boxes in UML.

The inheritance relation is shown by a line ending in a hollow triangle that goes from the more specific class to a more general class.

Figure 11.2 *UML diagram for inheritance*



[Access the text alternative for slide images.](#)

General Idea Part 3

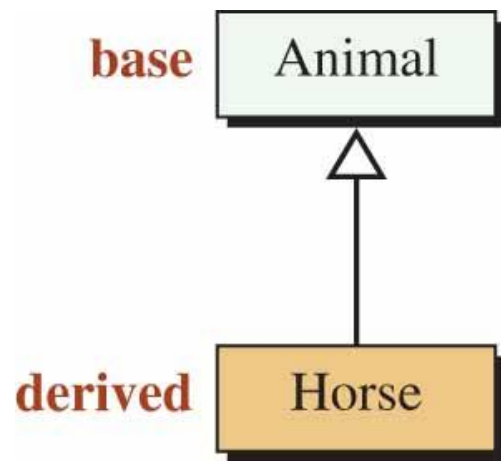
In C++, the most general class is called the *base class* and a more specific class is called the *derived class*.

A more general class is also known as a superclass; a more specific class is also known as a subclass.

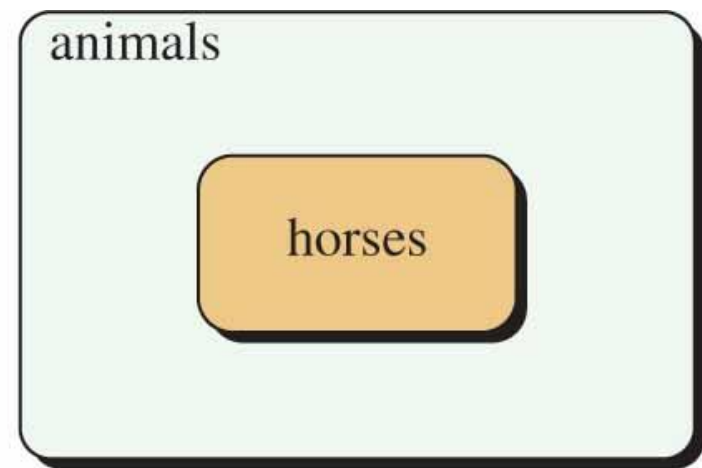
Inheritance Part 1 ₍₁₎

From a UML diagram, we can derive the relationship between the set of objects in an inheritance as shown in Figure 11.3, which shows that the set of horses is smaller than the set of animals.

Figure 11.3 *Classes and objects in inheritance hierarchy*



(a) Classes



(b) Objects

Inheritance Part 2 ₍₁₎

It should be obvious that a specific concept needs to have the characteristics of the general concept, but it can have more.

In other words, a horse should first be an animal then a horse.

That is why C++ says that a derived class extends its base class.

The term extends here means the derived class needs to have all of the data members and member functions defined in the base class (with the exception of constructors, destructor, and assignment operators that need to be redefined), and it can create new data members and member functions.

The derived class inherits all members (with some exceptions) from the base class, and it can add to them.

Inheritance Types Part 2

To create a derived class from a base class, we have three choices in C++:

private inheritance.

protected inheritance.

public inheritance.

To show the type of the inheritance we want to use, we insert a colon after the class, followed by one of the keywords (*private*, *protected*, or *public*).

Inheritance Types Part 3

Figure 11.4 shows these three types of inheritance in which B is the base class and D is the derived class.

Figure 11.4 *Inheritance types*

```
class D : public B
{
    ...
};
```

Public inheritance

```
class D : protected B
{
    ...
};
```

Protected inheritance

```
class D : private B
{
    ...
};
```

Private inheritance

Public Inheritance Part 1 (1)

Although the default inheritance type is private inheritance, the most common, by far, is public inheritance.

The other two types of inheritance are rarely used.

The most common use of inheritance is public inheritance.

Public Inheritance Part 2₍₁₎

Example 11.1

We design two classes, *Person* and *Student*, in which the class *Student* inherits from the class *Person*.

We know that a student is a person.

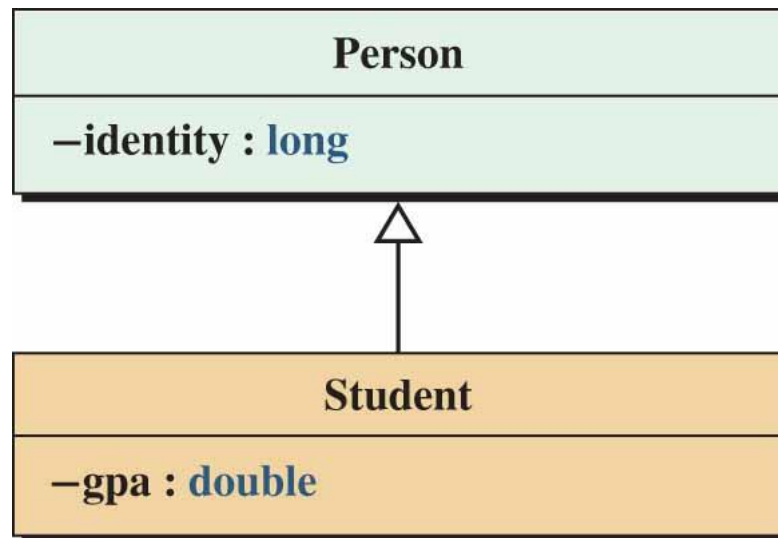
We assume that the *Person* class uses only one data member: *identity*.

We also assume that the *Student* class needs two data members: *identity* and *gpa*.

However, since the *identity* data member is already defined in the class *Person*, it does not need to be defined in the class *Student* because of inheritance.

Public Inheritance Part 3₍₁₎

Figure 11.5 *Two classes in inheritance relationship*



Notes:

The type of data members is shown after the member name separated by a colon.

The minus signs define the visibility of data members as private.

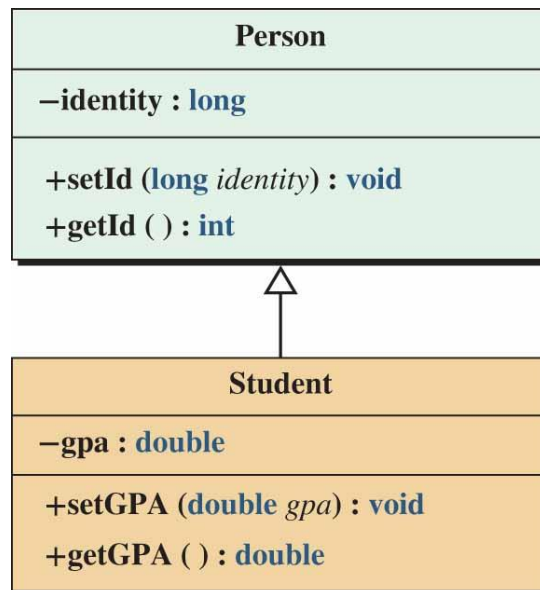
We can immediately see the advantage of inheritance.
The Student class uses the data member of the Person class and adds one data member of its own.

Public Inheritance₁

Example 11.2

We add some member functions to the two classes. We ignore the constructors and destructor at this point because they are not inherited; we assume the synthetic ones are used. We add one *getter* and one *setter* function to each class (Figure 11.6).

Figure 11.6 *Classes with data members and member functions*



Notes:

The type of data members and member functions is shown after the member name separated by a colon.

The minus signs define the visibility of data members as private; the plus signs define the visibility of the member functions as public.

Public Inheritance Part 1₍₂₎

Program 11.1 Public inheritance

```
1  /*****
2  * The program shows how we can let the class Student inherit *
3  * from the class Person because a student is a person.      *
4  *****/
5
6  #include <iostream>
7  #include <cassert>
8  #include <string>
9  using namespace std;
10
11 /*****
12 * The class definition for the Person class                  *
13 *****/
14 class Person
15 {
16     private:
17         long identity;
18     public:
19         void setId (long identity);
20         long getId( ) const;
```

Public Inheritance Part 2₍₂₎

Program 11.1 Public inheritance

```
21  };
22  /*****
23   * The definition of setId function in the Person class      *
24   *****/
25  void Person :: setId (long id)
26  {
27      identity = id;
28      assert (identity >= 100000000 && identity <= 999999999) ;
29  }
30
31  /*****
32   * The definition of the getId function in the Person class  *
33   *****/
34  long Person :: getId () const
35  {
36      return identity;
37  }
38
39  /*****
40   * The class definition for the Student class                *
```

Public Inheritance Part 3₍₂₎

Program 11.1 Public inheritance

```
41  *****/
42
43  class Student : public Person
44  {
45      private:
46          double gpa;
47      public:
48          void setGPA (double gpa);
49          double getGPA () const;
50  };
51
52  /**/
53  * The definition of setGPA function in Student class *
54  *****/
55  void Student :: setGPA (double gp)
56  {
57      gpa = gp;
58      assert (gpa >=0 && gpa <= 4.0);
59  }
60
```

Public Inheritance Part 4

Program 11.1 Public inheritance

```
61  /*****
62  * The definition of getGPA function in Student class      *
63  *****/
64  double Student :: getGPA() const
65  {
66      return gpa;
67  }
68
69
70  /*****
71  * The application function (main) that uses both classes  *
72  *****/
73  int main ( )
74  {
75      // Instantiation and use of a Person object
76      Person person;
77      person.setId (111111111L);
78      cout << "Person Information: " << endl;
79      cout << "Person's identity: " << person.getId ( );
80      cout << endl << endl;
```

Public Inheritance Part 5

Program 11.1 Public inheritance

```
81 // Instantiation and use of a Student object
82 Student student;
83 student.setId (222222222L);
84 student.setGPA (3.9);
85 cout << "Student Information: " << endl;
86 cout << "Student's identity: " << student.getId() << endl;
87 cout << "Student's gpa: " << student.getGPA();
88 return 0;
89 }
```

Run:

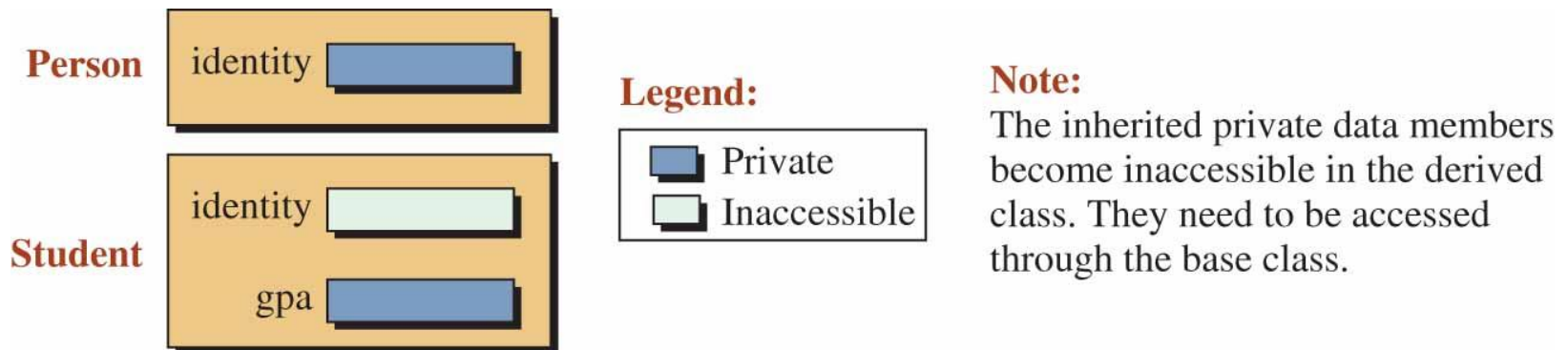
```
Person Information:
Person's Identity: 111111111

Student Information:
Student's identity: 222222222
Student's gpa: 3.9
```

Private Data Members

As Figure 11.7 shows, the base class object has only one data member, but the derived class object has two data members: one inherited and one created.

Figure 11.7 *Private data members in public inheritance*

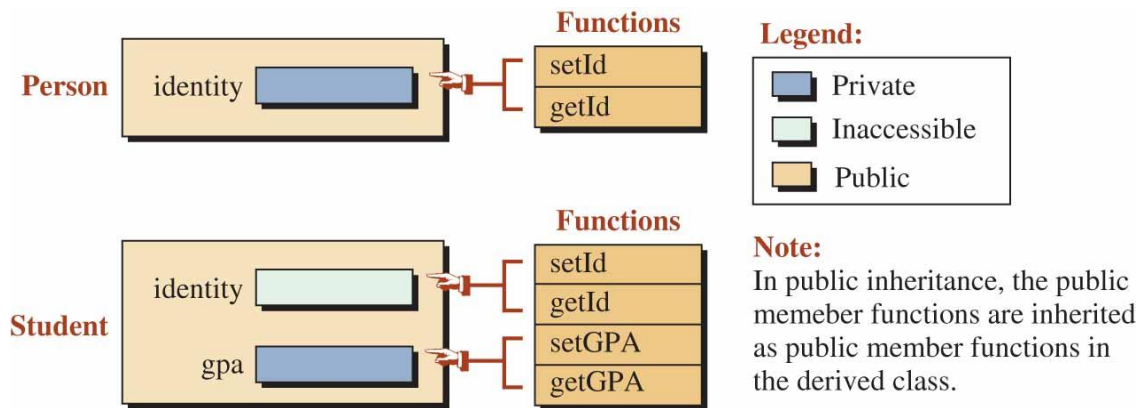


A private member in the base class becomes an inaccessible (hidden) member in the derived class.

Public Member Functions

Figure 11.8. adds public member functions to Figure 11.7. We can see that the base class has only two member functions while the derived class has four member functions, two inherited and two newly defined in the derived class.

Figure 11.8 *Public member functions in public inheritance*



A public member in the base class becomes a public member in the derived class.

Accessing Private Data Members

Now let us show how we access private data members in each class:

- In the base class, we can access the private data members through the public member function defined in this class (*setId* and *getId*).
- In the derived class, we need two groups of public member functions:
 - a. To access the inherited private data members, which are hidden, we use the public member functions defined in the base class (*setId* and *getId*).
 - b. If we need to access a data member defined in the derived class, we use the member functions defined in the derived class (*setGPA* and *getGPA*).

Functions with Same Names in Different Classes₁

In the two previous classes, Person and Student, we have used two member functions for setting data members (*setId* and *setGPA*) and two member functions for getting data members (*getId* and *getGPA*).

Can we use two functions with the same name, one in the base class and the other in the derived class?

In other words, can we have a function named *set* in the base class and another function also named *set* in the derived class?

Functions with Same Names in Different Classes₂

Similarly, can we have a function named *get* in the base class and another function also named *get* in the derived class?

The answer to both questions is positive, but we need to use two different concepts: *overloaded* and *overridden* functions.

To use the same name for a function in the base and derived classes, we need overloaded or overridden member functions.

11.1.2 Continued

Overloaded Member Functions.

Overloaded functions are two functions with the same name but with two different signatures.

Overloaded functions can be used in the same or different classes without being confused with each other.

We can have two functions named *set*, one in the base class and the other in the derived class with the following prototypes:

```
// In Person class  
void set(long identity);
```

```
// In Student class  
void set(double gpa);
```

Overridden Member Functions

If the signature of the two functions with the same name is the same, we have overridden member functions as shown below.

```
// In Person class  
long get() ;
```

```
// In Student class  
double get() ;
```

Class Scope Part 1

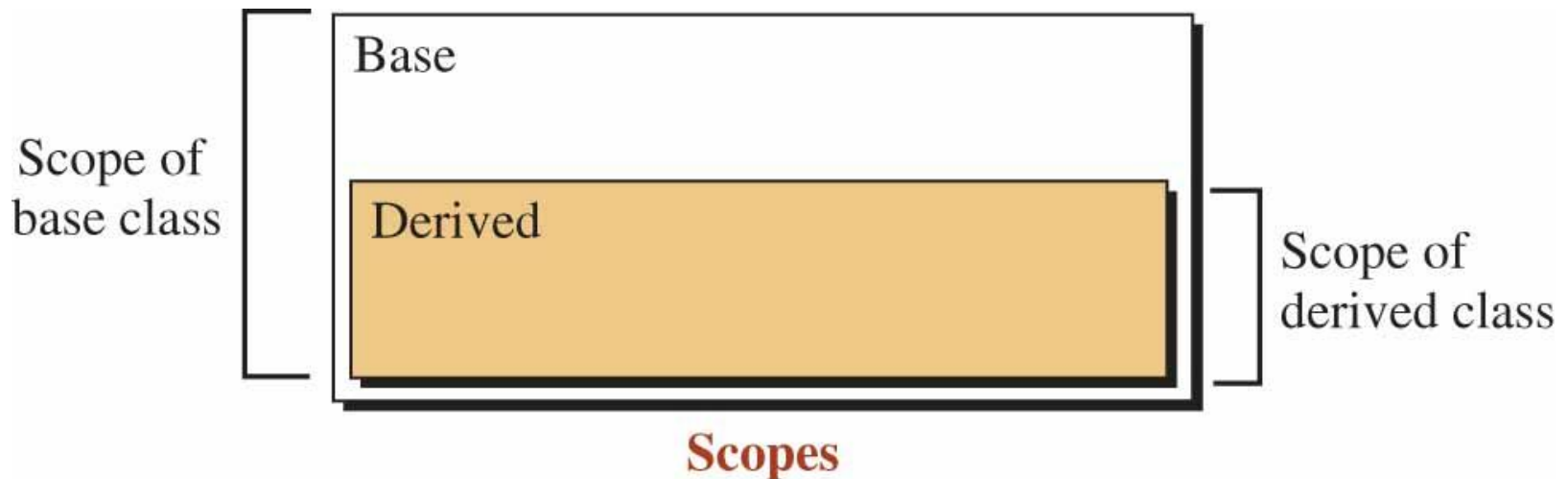
We can better understand the rules of inheritance if we know the scope to which the base and the derived classes belong.

The base class has its own class scope and the derived class has its own.

However, the scope of the derived class is enclosed in the scope of the base class.

Class Scope Part 2

Figure 11.9 *The scope of base and derived class*



The scope of the two classes shows that every name defined in the base class is also visible in the derived class, but the reverse is not true.

Class Scope Part 3

How can the system distinguish which function we are calling?

We need to remember that we do not call member functions in a class just by name, we let the instance call the appropriate function.

```
// Using Person object  
person.set(111111111L) ;  
person.get() ;
```

```
// Using Student object  
student.set(3.7) ;  
student.get() ;
```

Class Scope Part 4

The scope tells us how a compiler invokes a function based on the following rules:

- a.** The compiler tries to find a matching function (using names and parameters) that belongs to the class of the object that has invoked the function.
- b.** If no matching function is found, the compiler looks at the functions inherited from the superclass.
- c.** If no match is found, the search continues until the base class is reached.

Delegation Of Duty Part 1

An overloaded or overridden member function in a derived class can delegate part of its operation to a member function in a class in a higher level by calling the corresponding member function.

This is easily done with *void* member functions; it is more complicated with value returning functions because two values need to be returned which requires a *struct* or a *class* object.

We concentrate on the *void* member function in this case.

Delegation Of Duty Part 2

```
// Using Person object
void Person :: set(long id)
gp)
{
    identity = id;
}

void Person :: print()
{
    cout << name;
}
```

```
// Using Student object
void Student :: set(long id, double
gpa)
{
    Person :: set(id); // Delegation
    gpa = gp;
}

void Student :: print()
{
    Person :: print(); // Delegation
    cout << gpa;
}
```

Members Not Inherited : Invocation Part 1

There are five member functions that are not inherited in the derived class:

- 1. default constructor**
- 2. parameter constructor**
- 3. copy constructor**
- 4. destructor**
- 5. assignment operator.**

We postpone the discussion of the assignment operator until a future chapter, but we discuss the other four in this chapter.

Members Not Inherited : Invocation Part 2

Constructors, destructor, and assignment operators are not inherited; they need to be redefined.

The constructor of the derived class cannot initialize the data members of the base class because they are hidden in the derived class.

Similarly, the destructor of a derived class cannot delete the data members of the base class because they are hidden in the derived class.

Members Not Inherited : Invocation Part 3

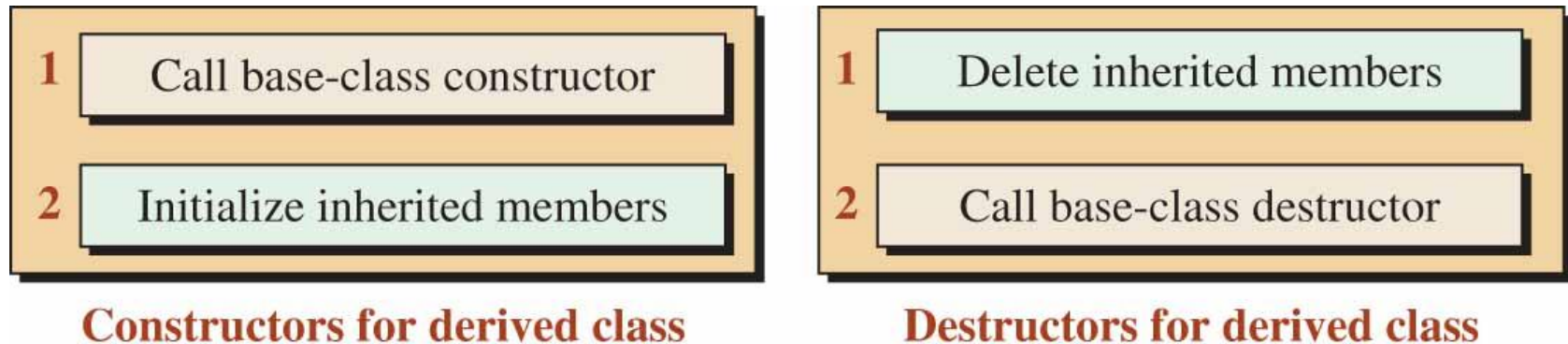
The constructor of the derived class invokes the constructor of the base class in its initialization and then initializes the data members of the derived class.

Similarly, the destructor of the derived class first deletes the data members of the derived class and then calls the destructor of the base class.

Constructor and Destructor in Inheritance

Part 1

Figure 11.10 *Constructor and destructor in inheritance*



Note that the order of activities in a constructor and destructor are reverse of each other.

We also should know that since the destructor is called by the system, not the user, the activities are done in the background unless the object uses pointers or files that may need the intervention of the user.

Definition of Constructors and Destructor

Part 2

Example 11.4

Table 11.1 shows the default constructor, parameter constructor, copy constructor, and destructor for our Person and Student classes side by side to see the difference.

Table 11.1 *Definition of Constructors and destructor*

Base Class	Derived Class
<pre>// Default Constructor Person :: Person() : identity(0) { }</pre>	<pre>// Default Constructor Student :: Student() : Person(0), gpa(0.0) { }</pre>
<pre>// Parameter Constructor Person :: Person (long id) identity(id), { }</pre>	<pre>// Parameter Constructor Student :: Student (long id, double gp) : : Person(id), gpa(gp) { }</pre>

Definition of Constructors and Destructor

Part 3

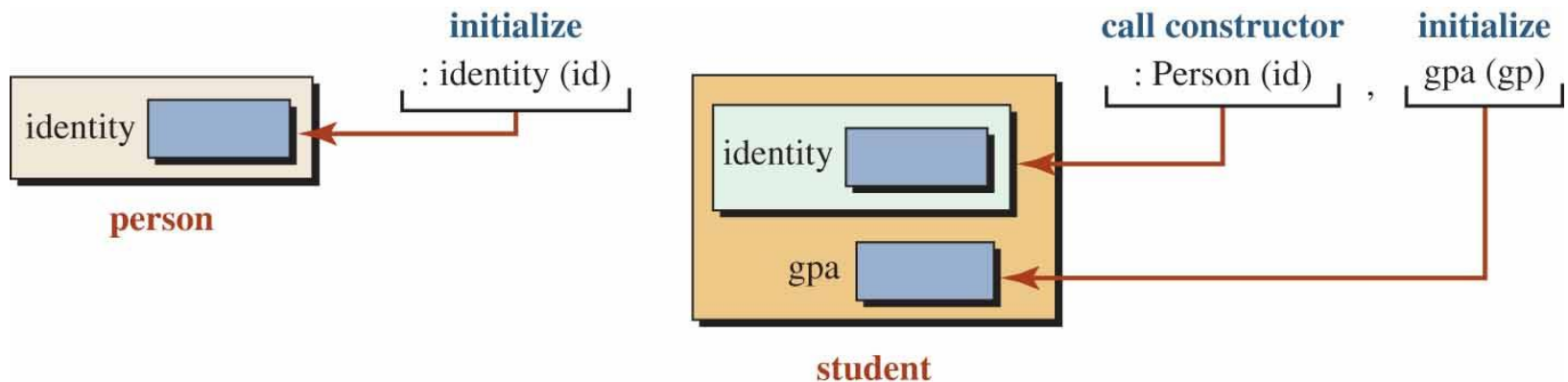
Table 11.1 Definition of Constructors and destructor

Base Class	Derived Class
<pre>// Copy Constructor Person :: Person(const Person& obj) : identity(obj.identity) : { }</pre>	<pre>// Copy Constructor Student :: Student(const Student& st) : Person(st) , gpa(std.gpa) { }</pre>
<pre>// Destructor Person :: ~Person() { }</pre>	<pre>// Destructor Student :: ~Student() { }</pre>

Constructing Objects

Figure 11.11 shows how the base class and the derived class objects are constructed using the parameter constructor.

Figure 11.11 *Constructing objects of type Person and Student*



Delegation Versus Invocation

Delegation and invocation are different concepts and are done differently.

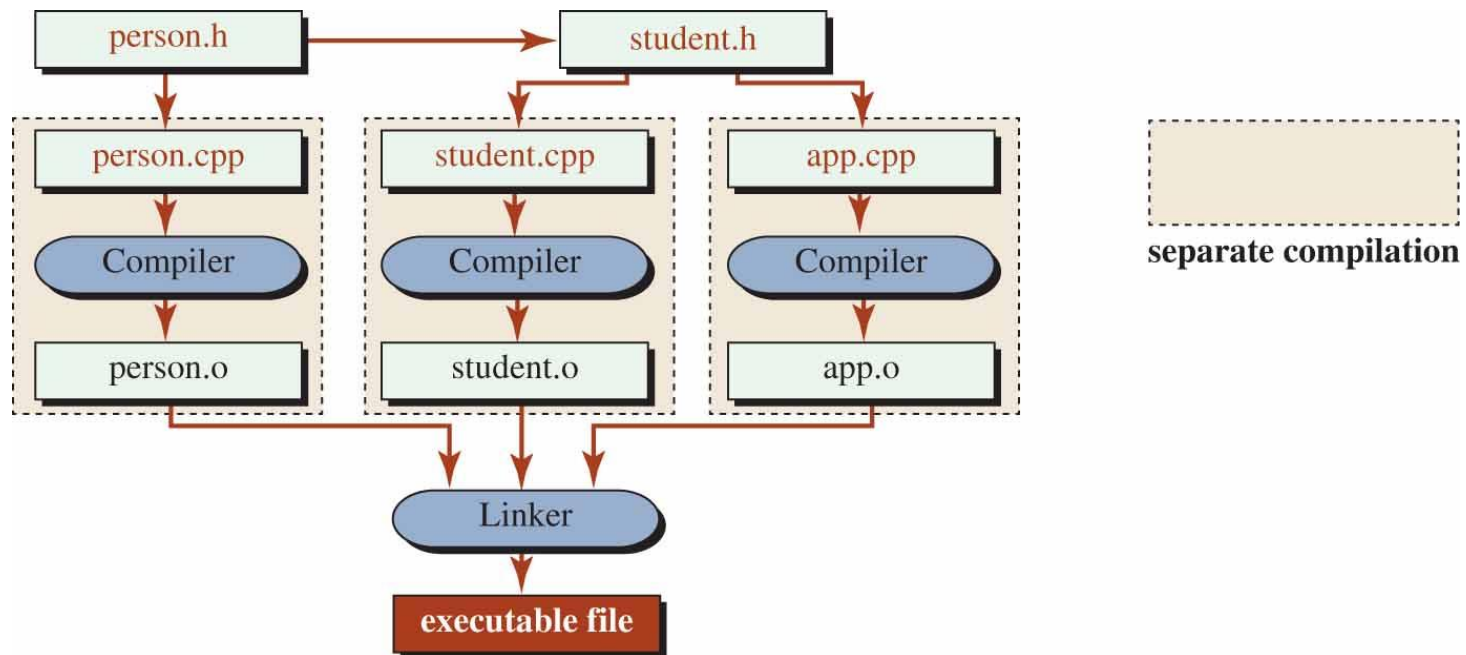
In delegation, a derived member function delegates part of its duty to the base class using the class resolution operator (::).

In invocation, the constructor of a derived class calls the constructor of the base class during initialization, which does not require the class resolution operator.

Separate Compilation

Figure 11.12 shows how separate compilation work using inheritance.

Figure 11.12 *Separate compilation*



Using Separate Compilation

An Example.

We give the interface, implementation, and application files for our simple inheritance example with constructors, destructor, and other functions.

We have added more data fields, but we eliminated the set and get functions and used only the print function to print the information for a person or a student.

The interface and implementation files for the Person Class.

We first give the interface and implementation files for the Person class. Program 11-2 and Program 11-3.

Using Separate Compilation Part 1

Program 11.2 File *person.h*

```
1  /*****
2  * The interface file for the Person class *
3  *****/
4
5  #ifndef PERSON_H
6  #define PERSON_H
7  #include <cassert>
8  #include <iostream>
9  #include <iomanip>
10 using namespace std;
11
12 class Person
13 {
14     private:
15         long identity;
16     public:
17         Person ();
18         Person (long identity);
19         ~Person ();
20         Person (const Person& person);
```

Using Separate Compilation Part 2

Program 11.2 File *person.h*

```
21         void print () const;  
22     };  
23 #endif
```

Using Separate Compilation Part 3

Program 11.3 File *person.cpp*

```
1  /*****
2  * The implementation file for the Person class          *
3  *****/
4  #include "person.h"
5
6  // Default constructor
7  Person :: Person ()
8  : identity (0)
9  {
10 }
11 // Parameter constructor
12 Person :: Person (long id)
13 : identity (id)
14 {
15     assert (identity >= 100000000 && identity <= 999999999);
16 }
17 // Copy constructor
18 Person :: Person (const Person& person)
19 : identity (person.identity)
20 {
```

Using Separate Compilation Part 4

Program 11.3 File *person.cpp*

```
21 }  
22 // Destructor  
23 Person:: ~Person()  
24 {  
25 }  
26 // Accessor member function  
27 void Person :: print () const  
28 {  
29     cout << "Identity: " << identity << endl;  
30 }
```


Interface File for Student Class

Program 11.4 File *student.h*

```
1  /*****
2  * The interface file for the Student class          *
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include "person.h"
7
8  class Student: public Person
9  {
10     private:
11         double gpa;
12     public:
13         Student ( );
14         Student (long identity, double gpa);
15         ~Student();
16         Student (const Student& student);
17         void print () const;
18 };
19 #endif
```

Implementation Files for Student Class

Part 1

Program 11.5 File *student.cpp*

```
1  /*****
2  * The implementation file for the Student class          *
3  *****/
4  #include "student.h"
5
6  // Default constructor
7  Student :: Student ()
8  : Person (), gpa (0.0)
9  {
10 }
11 // Parameter constructor
12 Student :: Student (long id, double gp)
13 : Person (id), gpa (gp)
14 {
15     assert (gpa >= 0.0 && gpa <= 4.0);
16 }
17 // Copy constructor
18 Student :: Student (const Student& student)
19 : Person (student), gpa (student.gpa)
20 {
```

Implementation Files for Student Class

Part 2

Program 11.5 File *person.cpp*

```
21  }
22  // Destructor
23  Student :: ~Student()
24  {
25  }
26  // Accessor member function
27  void Student :: print () const
28  {
29      Person :: print ();
30      cout << "GPA: " << fixed << setprecision (2) << gpa << endl;
31  }
```

Application File to Use the Classes Part 1

Program 11.6 shows a simple example of the application file.

The client can create any application file and use the Person and Student classes.

Application File to Use the Classes Part 2

Program 11.6 File *app.cpp*

```
1  /*****
2  * The application to test the Person and Student classes      *
3  *****/
4  #include "student.h"
5
6  int main ( )
7  {
8      // Instantiation and using a Person object
9      Person person (11111111);
10     cout << "Information about person: " << endl;
11     person.print ();
12     cout << endl;
13     // Instantiation and using a Student object
14     Student student (22222222, 3.9);
15     cout << "Information about student: " << endl;
16     student.print ();
17     cout << endl;
18     return 0;
19 }
```

Application File to Use the Classes Part 3

Program 11.6 *File app.cpp*

Run:

Information about person:

Identity: 111111111

Information about student:

Identity: 222222222

GPA: 3.90

More About Public Inheritance Part 1

Protected Members.

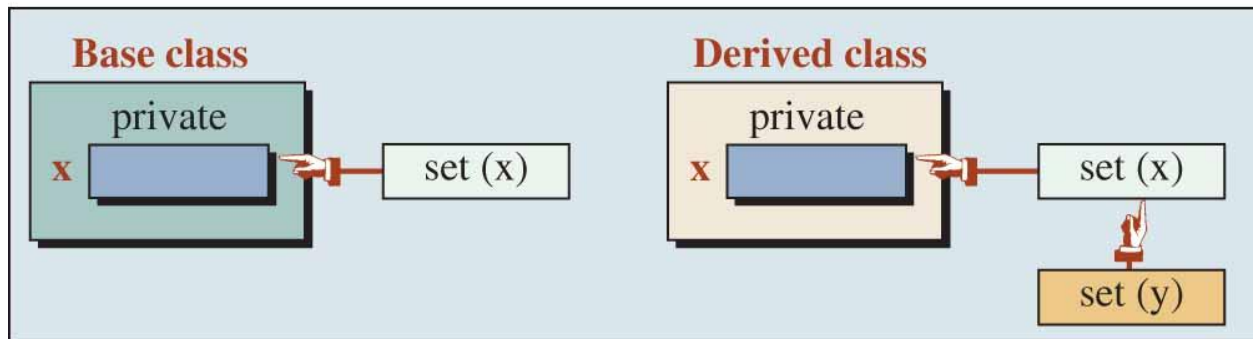
C++ also defines another specifier for a member, *protected*. A protected member is accessible in the derived class and all classes derived from the derived class.

The functions defined in the derived class can easily access a protected member.

More About Public Inheritance Part 2

Figure 11.13 *Private versus protected member*

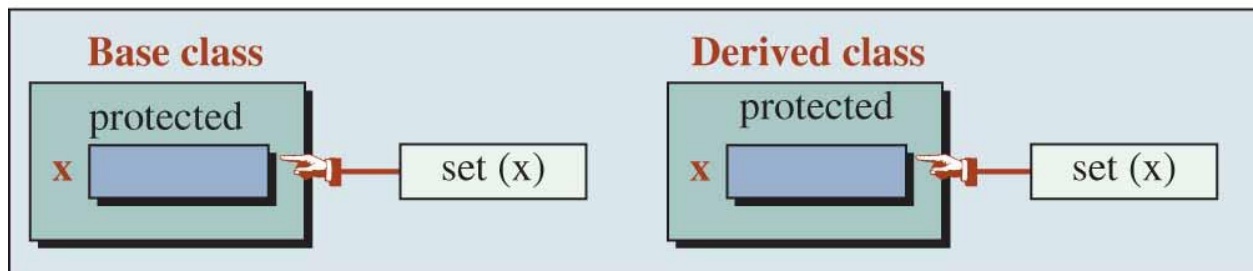
Using private data member in inheritance



Note:

Parameter y is a dummy one to be used to set x.

Using protected data member in inheritance



More About Public Inheritance Part 3

The figure from previous slide shows two cases.

In the first case, we have only one private data member and one public member function.

When we use derivation, the private member x is hidden.

Although the function `set(x)` is public and seen in the derived class, it cannot be called directly because the parameter x cannot be seen.

More About Public Inheritance Part 4

We need to create another function, with another dummy variable *y* to access *x* as shown below:

```
void Derived :: set(int y)
{
    Base :: set(y);    // Calling the inherited function
}
```

If we have defined the data member as protected, it is seen in the derived class and we can use `set(x)` directly without having to create another function with a dummy variable.

Using Private Data Members

Using private data members enforces the concept of encapsulation.

When we use private data members, data are hidden to entities outside of the class and also to derived classes.

On the other hand, using private data members means creating more code in the derived class.

Therefore, the advantage of private data members is stronger encapsulation; its disadvantage is creating more code in the derived class.

Using Protected Data Members

Using protected data members makes the coding of the derived classes simpler.

However, it breaks the idea of encapsulation.

Sometimes we need to use protected data members because the coding becomes very complicated if we use private data members.

Blocking Inheritance

Sometimes we may want to block inheritance. The C++ standard allows us to do so using the final modifier as shown below:

```
class First final
{
    ...
}
```

We can also use the final modifier to stop the inheritance anywhere in the hierarchy.

```
class First
{
    ...
}
class Second final : public First
{
    ...
}
```

Liakhov Substitution Principle Part 1

One of design principles that is sometimes ignored is called the Liskov Substitution Principle (LSP), developed by Barbara Liskov.

This principle says that an object of a superclass must always be substitutable by an object of a subclass without altering any of the properties of the superclass.

We may think of an object as a special kind of another object in real life, but in object-oriented programming, we need to check LSP before writing the code.

Liskov Substitution Principle Part 2

Example 11.5

To better understand the Liskov Substitution Principle, let us consider two objects: square and rectangle.

In real life, we may say that a square is a special kind of a rectangle in which the length and the width are the same.

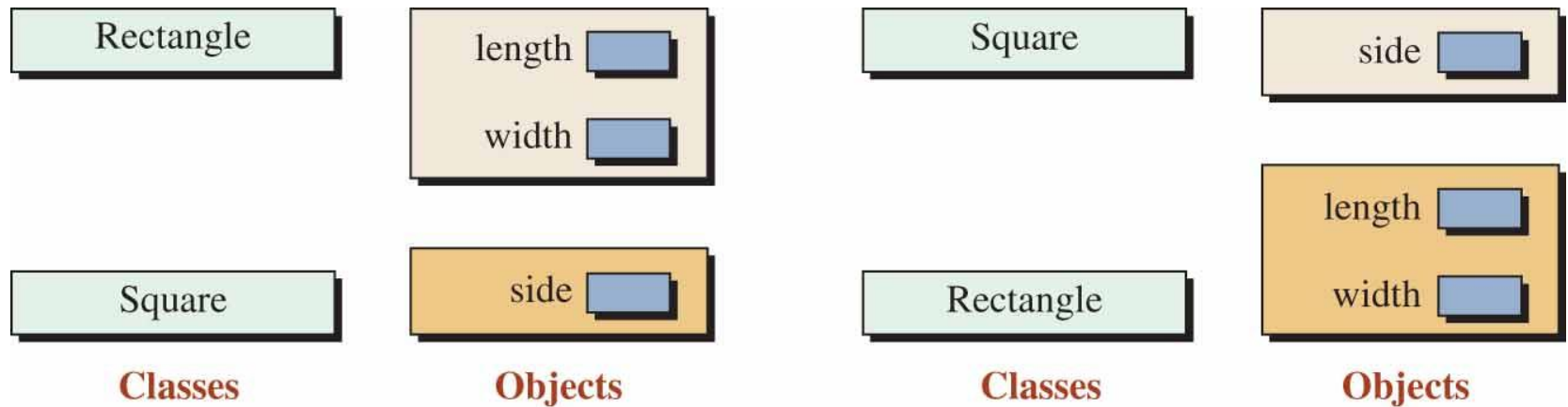
We may also say that a rectangle is a square in which the length and sides are not of the same size.

Neither of these two definition comply with the "*is-a*" relationship in object-oriented programming.

To understand the reason let us examine both approaches.

Liskov Substitution Principle Part 3

Figure 11.14 *Liskov Substitution Principle*



Liskov Substitution Principle Part 4

In the left case, we may think that the *Square* class can be derived from the *Rectangle* class because a square is a special kind of a rectangle.

However, we see that an object of type *Square* has only one data member instead of two.

An object of type *Square* cannot encapsulate an object of type *Rectangle* so it is not substitutable.

Liskov Substitution Principle Part 5

In the right case, an object of the Rectangle class has more data member than an object of the Square class, but none of them can be copied from the Square class.

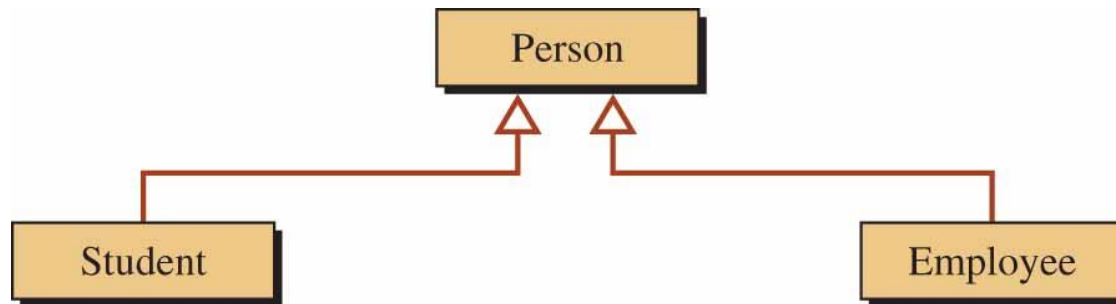
In other words an object of Rectangle class has no relationship with an object of Square class and cannot be substituted for it.

Inheritance Tree

In C++, we can have an inheritance tree. For example, we can have two classes that inherit from the Person class: Student and Employee.

It is clear that a student is a person and an employee is also a person.

Figure 11.15 *Inheritance Tree*



Example

Let us create interface files, implementation files, and application files for classes in Figure 11.15.

Interface File to Define the Person Class

Part 1

Program 11.7 File *person.h*

```
1  /*****
2  * The interface file to define the Person class          *
3  *****/
4  #ifndef PERSON_H
5  #define PERSON_H
6  #include <iostream>
7  #include <string>
8  using namespace std;
9
10 // Definition of Person class
11 class Person
12 {
13     private:
14         string name;
15     public:
16         Person (string nme);
17         ~Person ();
18         void print () const;
19 };
20 #endif
```

Implementation File for the Person Class

Program 11.8 File *person.cpp*

```
1  /*****
2  * The implementation file for the Person class          *
3  *****/
4  #include "person.h"
5
6  // Constructor for Person class
7  Person :: Person (string nm)
8  :name (nm)
9  {
10 }
11 // Destructor for Person class
12 Person :: ~Person()
13 {
14 }
15 // Definition of print member function
16 void Person :: print () const
17 {
18     cout << "Name: " << name << endl;
19 }
```

Interface File to Define the Student Class

Program 11.9 File student.h

```
1  /*****
2  * The interface file to define the Student class          *
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include "person.h"
7
8  // Definition of the Student class
9  class Student : public Person
10 {
11     private:
12         string name;
13         double gpa;
14     public:
15         Student (string name, double gpa);
16         ~Student ( );
17         void print ( ) const;
18 };
19 #endif
```

Implementation File for the Student Class

Program 11.10 File student.cpp

```
1  /*****
2  * The interface file to define the Employee class          *
3  *****/
4  #include "student.h"
5
6  // Constructor for Student class
7  Student :: Student (string nm, double gp)
8  :Person (nm), gpa (gp)
9  {
10 }
11 // Destructor for Student class
12 Student :: ~Student ()
13 {
14 }
15 // Definition of the print member function
16 void Student :: print () const
17 {
18     Person :: print();
19     cout << "GPA: " << gpa << endl;
20 }
```

Interface File to Define the Employee Class

Program 11.11 File employee.h

```
1  /*****
2  * The interface file to define the Student class          *
3  *****/
4  #ifndef EMPLOYEE_H
5  #define EMPLOYEE_H
6  #include "person.h"
7
8  // Definition of the Employee class
9  class Employee : public Person
10 {
11     private:
12         string name;
13         double salary;
14     public:
15         Employee (string name, double salary);
16         ~Employee ( );
17         void print ( ) const;
18 };
19 #endif
```


Implementation File for the Employee Class

Program 11.12 File *employee.cpp*

```
1  /*****
2   * The implementation file for the Employee class      *
3   *****/
4  #include "employee.h"
5
6  // Constructor for Employee class
7  Employee :: Employee (string nm, double sa)
8  :Person (nm), salary (sa)
9  {
10 }
11 // Destructor for Employee class
12 Employee :: ~Employee()
13 {
14 }
15 // Definition of print member function
16 void Employee :: print () const
17 {
18     Person :: print();
19     cout << "Salary: " << salary << endl;
20 }
```

Application File to Use Classes Part 1

Program 11.13 File *application.cpp*

```
1  /*****
2   * The application file to use classes *
3   *****/
4  #include "student.h"
5  #include "employee.h"
6
7  int main ()
8  {
9      // Instantiation and using an object of the Person class
10     cout << "Person: " << endl;
11     Person person ("John");
12     person.print ();
13     cout << endl << endl;
14     // Instantiation and using an object of the Student class
15     cout << "Student: " << endl;
16     Student student ("Mary", 3.9);
17     student.print ();
18     cout << endl << endl;
19     // Instantiation and using an object of the Employee class
20     cout << "Employee: " << endl;
```

Application File to Use Classes Part 2

Program 11.13 *File application.cpp*

```
21     Employee employee ("Juan", 78000.00);  
22     employee.print ();  
23     cout << endl << endl;  
24     return 0;  
25 }
```

Run:

Person:

Name: John

Student:

Name: Mary

GPA: 3.9

Employee:

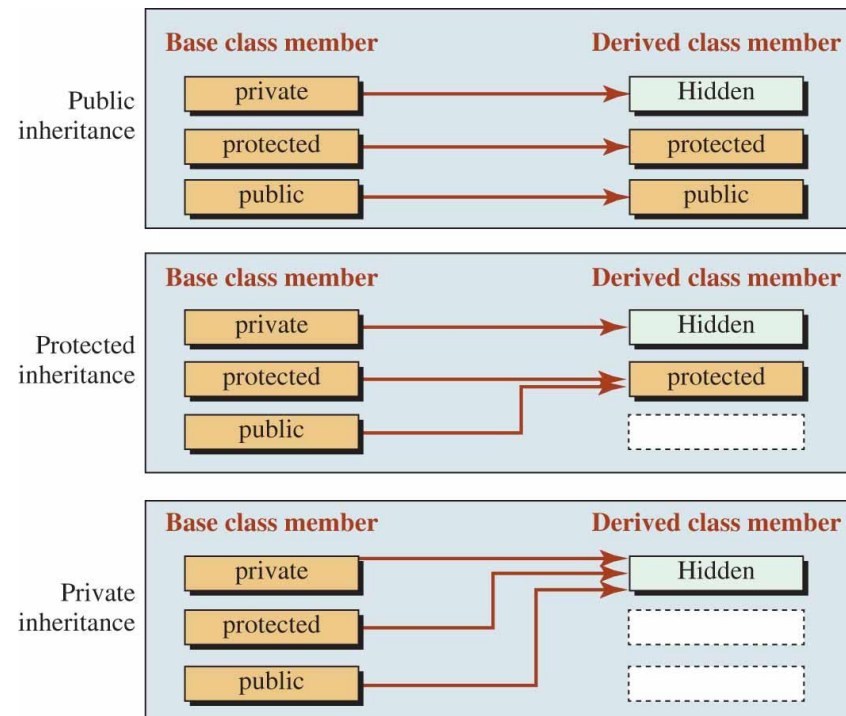
Name: Juan

Salary: 78000

Three Types of Inheritance

Although, public inheritance is by far the most common type of derivation, C++ allows us to use two other types of derivation: *private* and *protected*.

Figure 11.16 *Inheritance types*



[Access the text alternative for slide images.](#)

Public Inheritance₂

Public Inheritance.

Public inheritance is what we use most of the time.

This type of derivation defines an *is-a* relationship between the base class object and the derived class object because the public interface of the base class becomes a public interface of the derived class.

Protected and Private Inheritance

Protected Inheritance.

This type of inheritance is rare and virtually never used.

Private Inheritance.

Private inheritance is much less common than public inheritance, but it has some applications.

Public members of the base class become private members in the derived class.

This property allows inherited implementation (code reuse).

Inheritance Part 1 ₍₂₎

Assume that we have written the code for a class. Now we want to write the code for another class, but there is no "*is-a*" relationship between the two.

However, we notice that some of the public members of the first class can help us code some of the public members of the new class.

We inherit the second class privately from the first class to use some of its functions.

Inheritance Part 2 ₍₂₎

```
void First :: functionFirst()
{
    ...
}
void Second :: functionSecond()
{
    First : FunctionFirst() // Use FunctionFirst to do some of the job
    ...                    // Add more code to the rest of the job
}
```

**Private inheritance does not define a type inheritance;
it defines an implementation inheritance.**

ASSOCIATION PART 1

The second type of relationship is *association*. As a matter of fact, more programs are being developed today that use association rather than inheritance.

An association between two classes shows a relationship.

For example, we can define a class named *Person* and another class named *Address*.

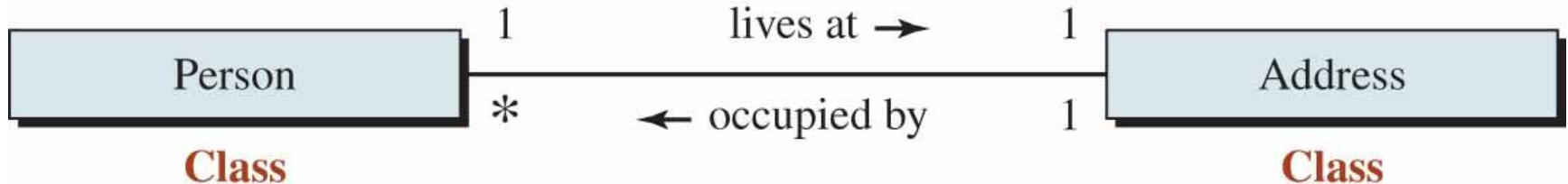
An object of the type *Person* may be related to an object of type *Address*: a person lives in an address and the address is occupied by a person.

The *Address* class is not inherited from the *Person* class; neither the other way.

ASSOCIATION PART 2

A relationship of this type is shown in UML diagrams as a solid line between two classes.

Figure 11.17 *An association relationship*



An association diagram also shows the type of relationship using an arrow head and text in the direction of the corresponding class.

ASSOCIATION PART 3

Another piece of information represented in an association diagram is multiplicity.

Multiplicity defines the number of objects that take part in the association.

Table 11.2 shows different types of multiplicity.

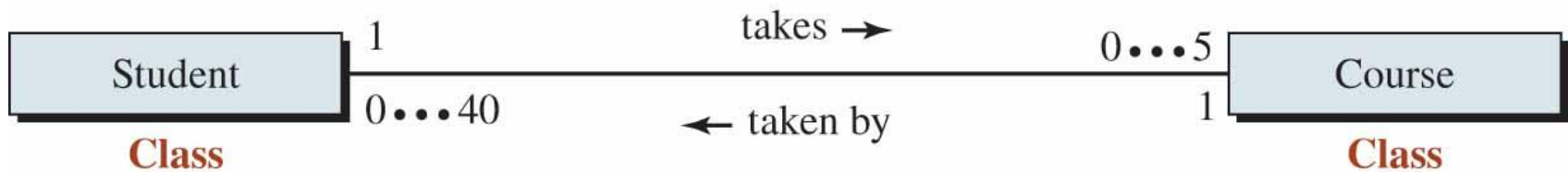
Table 11.2 *Multiplicity in association diagrams*

Key	Interpretation
n	Exactly n objects
$*$	Any number of objects including none
$0 \dots 1$	Zero or one object
$n \dots m$	A range from n to m objects
n, m	n or m objects

ASSOCIATION PART 4

As another example, we define the association relationship between students and courses they take in Figure 11.18.

Figure 11.18 *Association between courses and students*



For example, a Student object can have a list of five course names and a Course object can have a list of forty student names.

Aggregation Part 1

Aggregation is a special kind of association in which the relationship involves ownership.

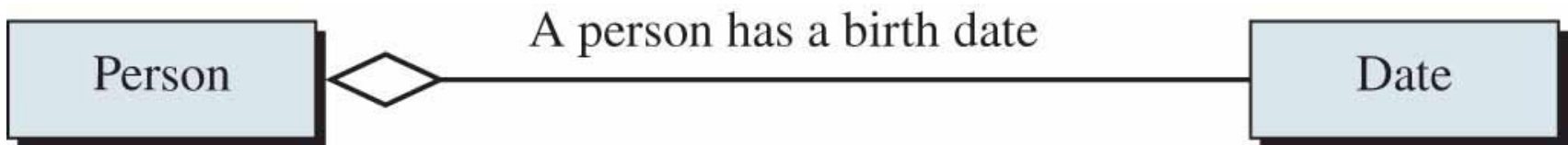
In other words, it models the "*has-a*" relationship. One class is called an aggregator and the other an aggregatee.

In other words, an object of the aggregator class contains one or more objects of the aggregatee class.

Aggregation Part 2

Figure 11.19 shows the UML diagram for aggregation. Note that the symbol for aggregation is a hollow diamond placed at the site of the aggregator.

Figure 11.19 *Example of aggregation relationship*



An aggregation is a one-to-many relationship from the aggregator to the aggregatee.

In an aggregation, the lifetime of the aggregatee is independent of the lifetime of the aggregator.

Interface File for date.h Class

Program 11.14 The file date.h

```
1  /*****
2  * The interface file for date.h class
3  *****/
4  #ifndef DATE_H
5  #define DATE_H
6  #include <iostream>
7  #include <cassert>
8  using namespace std;
9
10 class Date
11 {
12     private:
13         int month;
14         int day;
15         int year;
16     public:
17         Date (int month, int day, int year);
18         ~Date ();
19         void print() const;
20 };
```

Interface File for date.h Class Part 2

Program 11.14 *The file date.h*

```
21 #endif
```


Implementation of Functions in the Date Class Part 2 ₍₁₎

Program 11.15 File date.cpp

```
1  /*****
2  * The implementation of functions in the Date class      *
3  *****/
4  #include "date.h"
5
6  // Parameter constructor
7  Date :: Date (int m, int d, int y)
8  : month (m), day (d), year (y)
9  {
10     if ((month < 1) || (month > 12))
11     {
12         cout << "Month is out of range. ";
13         assert (false);
14     }
15     int daysInMonths [13] = {0, 31, 28, 31, 30, 31, 30, 31,
16                             31, 30, 31, 30 ,31};
17     if ((day < 1) || (day > daysInMonths [month]))
18     {
19         cout << "Day out of range! ";
20         assert (false);
```

Implementation of Functions in the Date Class Part 2 ₍₂₎

Program 11.15 File date.cpp

```
21     }
22     if ((year < 1900) || (year > 2099))
23     {
24         cout << "Year out of range! " ;
25         assert (false);
26     }
27 }
28 // Destructor
29 Date :: ~Date ()
30 {
31 }
32 // Print member function
33 void Date :: print() const
34 {
35     cout << month << "/" << day << "/" << year << endl;
36 }
```

Interface File for the Person Class

Program 11.16 File *person.h*

```
1  /*****
2  * The interface file for the Person class      *
3  *****/
4  #ifndef PERSON_H
5  #define PERSON_H
6  #include "date.h"
7
8  // Definition of the Person class
9  class Person
10 {
11     private:
12         long identity;
13         Date birthDate;
14     public:
15         Person (long identity, Date birthDate);
16         ~Person ( );
17         void print ( ) const;
18 };
19 #endif
```

Implementation File for Person Concrete Class Part 1

Program 11.17 File *person.cpp*

```
1  /*****
2   * The implementation file for Person concrete class      *
3   *****/
4  #include "person.h"
5
6  // Constructor
7  Person :: Person (long id, Date bd)
8  : identity (id), birthDate (bd)
9  {
10     assert (identity > 111111111 && identity < 999999999);
11 }
12 // Destructor
13 Person :: ~Person ( )
14 {
15 }
16 // Print function
17 void Person :: print ( ) const
18 {
19     cout << "Person Identity: " << identity << endl;
20     cout << "Person date of birth: ";
```

Implementation File for Person Concrete Class Part 2

Program 11.17 File *person.cpp*

```
21     birthDate.print ();  
22     cout << endl << endl;  
23 }
```

Application File to Test the Person Class

Part 1

Program 11.18 File *app.cpp*

```
1  /*****
2  * The application file to test the Person class          *
3  *****/
4  #include "person.h"
5
6  int main ( )
7  {
8      // Instantiation
9      Date date1 (5, 6, 1980);
10     Person person1 (111111456, date1);
11     Date date2 (4, 23, 1978);
12     Person person2 (345332446, date2);
13     // Output
14     person1.print ( );
15     person2.print ( );
16     return 0;
17 }
```

Application File to Test the Person Class

Part 2

Program 11.18 *File app.cpp*

Run:

Person Identity: 111111456

Person date of birth: 5/6/1980

Person Identity: 345332446

Person date of birth: 4/23/1978

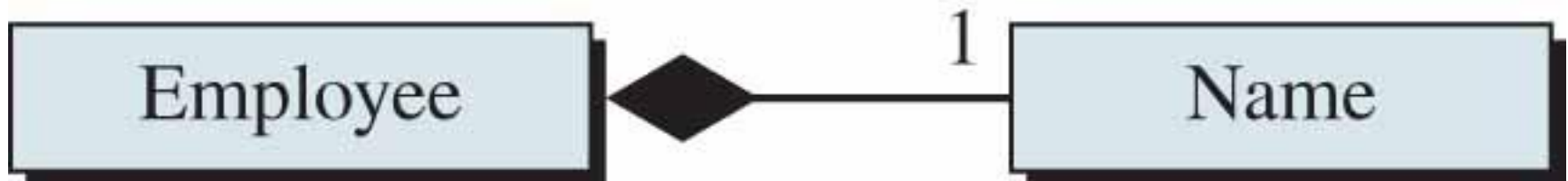
Composition

A composition is a special kind of aggregation in which the lifetime of the containee depends on the lifetime of the container.

For example, the relationship between a person and her name is an example of composition.

The name cannot exist without being the name of a person. Figure 11.20 shows the relationship between an employee and her name.

Figure 11.20 *Example of composition relationship*



Interface File for the Name Class Part 1

Program 11.19 File name.h

```
1  /*****
2  * The interface file for the Name class *
3  *****/
4
5  #ifndef NAME_H
6  #define NAME_H
7  #include <string>
8  #include <iostream>
9  #include <cassert>
10 using namespace std;
11
12 class Name
13 {
14     private:
15         string first;
16         string init;
17         string last;
18     public:
19         Name (string first, string init, string last);
20         ~Name ( );
```

Interface File for the Name Class Part 2

Program 11.19 *File name.h*

```
21     void print ( ) const;  
22 };  
23 #endif
```

Implementation File for Name Class Part 1

Program 11.20 File name.cpp

```
1  /*****
2  * The implementation file for Name class *
3  *****/
4  #include "name.h"
5
6  // Constructor
7  Name :: Name (string fst, string i, string lst)
8  :first (fst), init (i), last (lst)
9  {
10     assert (init.size () == 1);
11     toupper (first[0]);
12     toupper (init [0]);
13     toupper (last[0]);
14 }
15 // Destructor
16 Name :: ~Name ( )
17 {
18 }
19 // Print member function
20 void Name :: print ( ) const
```

Implementation File for Name Class Part 2

Program 11.20 *File name.cpp*

```
21 {  
22     cout << "Employee name: " << first << " " << init << ". ";  
23     cout << last << endl;  
24 }
```

Interface File for the Employee Class

Part 1

Program 11.21 File *employee.h*

```
1  /*****
2  * The interface file for the employee class      *
3  *****/
4  #ifndef EMPLOYEE_H
5  #define EMPLOYEE_H
6  #include "name.h"
7
8  class Employee
9  {
10     private:
11         Name name;
12         double salary;
13     public:
14         Employee (string first, string init, string last,
15                  double salary);
16         ~Employee ( );
17         void print ( ) const;
18 };
19 #endif
```

Implementation File for Employee Class

Part 1

Program 11.22 File *employee.cpp*

```
1  /*****
2  * The implementation file for Employee class      *
3  *****/
4
5  #include "employee.h"
6
7  // Constructor
8  Employee :: Employee (string fst, string i, string lst,
9                      double sal)
10 : name (fst, i, lst), salary (sal)
11 {
12     assert (salary > 0.0 and salary < 100000.0);
13 }
14 // Destructor
15 Employee :: ~Employee ( )
16 {
17 }
18 // Print member function
19 void Employee :: print ( ) const
20 {
```

Implementation File for Employee Class

Part 2

Program 11.22 *File employee.cpp*

```
21     name.print();  
22     cout << "Salary: " << salary << endl << endl;  
23 }
```

Application File to Test the Employee Class

Part 1

Program 11.23 *The file appl.cpp*

```
1  /*****
2  * The application file to test the Employee class      *
3  *****/
4  #include "employee.h"
5
6  int main ( )
7  {
8      // Instantiation
9      Employee employee1 ("Mary", "B", "White", 22120.00);
10     Employee employee2 ("William", "S", "Black", 46700.00);
11     Employee employee3 ("Ryan", "A", "Brown", 12500.00);
12     // Output
13     employee1.print ( );
14     employee2.print ( );
15     employee3.print ( );
16     return 0;
17 }
```


Application File to Test the Employee Class

Part 2

Program 11.23 *The file appl.cpp*

Run:

Employee name: Mary B. White

Salary: 22120

Employee name: William S. Black

Salary: 46700

Employee name: Ryan A. Brown

Salary: 12500

DEPENDENCY PART 1

The third type of relationship is dependency. Dependency is a weaker relationship than inheritance or association.

We say that dependency models the "*uses*" relationship.

Class A depends on class B if class A somehow uses class B. This happens when.

DEPENDENCY PART 2

Class A uses an object of type B as a parameter in a member function.

Class A has a member function that returns an object of type B.

Class A has a member function that has a local variable of type B.

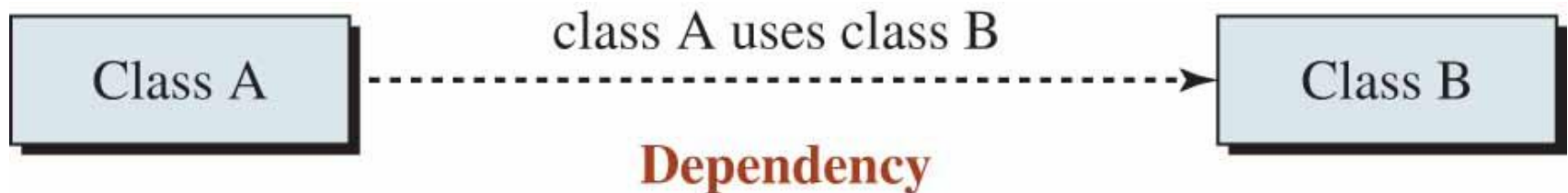
UML Diagrams Part 1

We use both UML class diagrams and UML sequence diagrams to show the dependencies.

UML Class Diagram.

Figure 11.21 shows an example of a dependency relationship in which class A depends on class B.

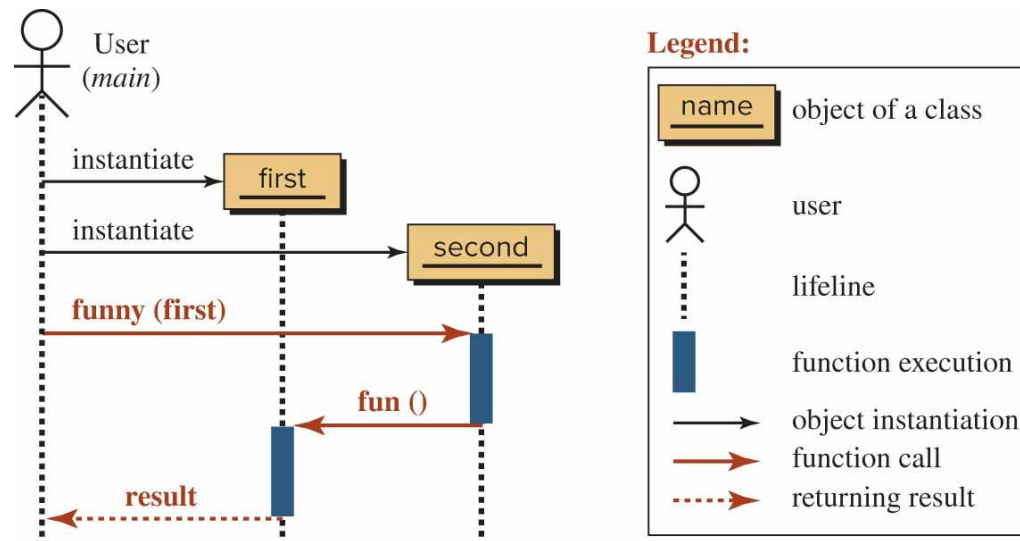
Figure 11.21 *UML class diagram for dependency*



UML Diagrams Part 2

A *sequence* diagram shows the interaction between objects. The *main* function and each object has a *lifeline* that shows the passing of time. The objects can be instantiated and their member function can be called.

Figure 11.22 *Sequence diagram for the previous example*



UML Diagrams Part 3

In the figure, the main function instantiates an object of the class First and an object of the class Second.

The *main* function calls the *funny* (...) member function of the class Second and passes an object of the class First as a parameter.

An object of the class Second can then call the *fun()* function of the class First using the name of the object that received from *main*.

A Comprehensive Example Part 1

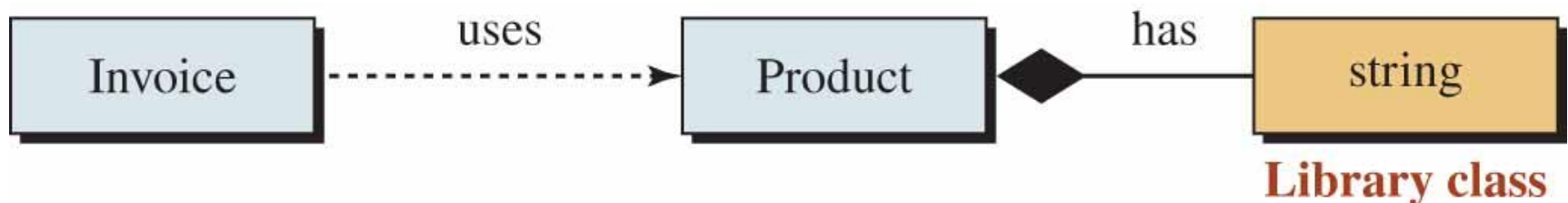
We use a comprehensive example to demonstrate the basics of dependency relationships.

Assume we want to create an invoice for the list of products sold.

We have a class named *Invoice* and a class named *Product*.

The class *Invoice* uses instances of the class *Product* as a parameter in one of its member functions (*add*).

Figure 11.23 *UML class diagram for Invoice and Product*



A Comprehensive Example Part 2

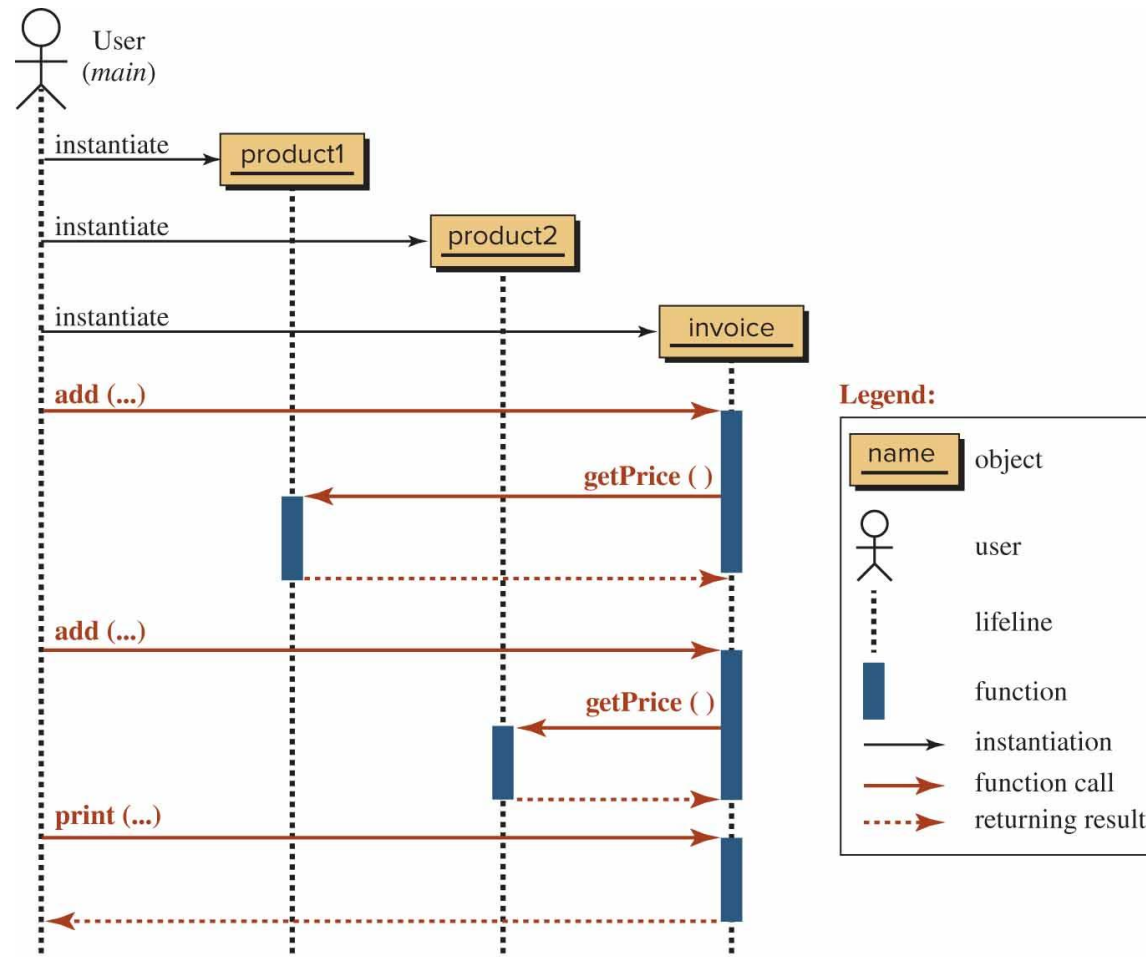
We show the sequence diagram for two products in Figure 11.24.

Note that the *main* function needs to instantiate two object of type Product and one object of type Invoice.

The *main* function then calls the *add* function in the Invoice class to add the products to the invoice, but it needs to get the price of each product from the corresponding object.

A Comprehensive Example Part 3

Figure 11.24 Sequence diagram for the invoice program



[Access the text alternative for slide images.](#)

Interface File for the Product Class

Program 11.24 File *product.h*

```
1  /*****
2  * The interface file for the Product class      *
3  *****/
4  #ifndef PRODUCT_H
5  #define PRODUCT_H
6  #include <string>
7  #include <iostream>
8  using namespace std;
9
10 class Product
11 {
12     private:
13         string name;
14         double unitPrice;
15     public:
16         Product (string name, double unitPrice);
17         ~Product ( );
18         double getPrice ( ) const;
19 };
20 #endif
```

Implementation File for Product Class

Program 11.25 File *product.cpp*

```
1  /*****
2  * The implementation file for Product class          *
3  *****/
4  #include "product.h"
5
6  // Constructor
7  Product :: Product (string nm, double up)
8  : name (nm), unitPrice (up)
9  {
10 }
11 // Destructor
12 Product :: ~Product ( )
13 {
14 }
15 // The getPrice member function
16 double Product :: getPrice ( ) const
17 {
18     return unitPrice;
19 }
```

Interface File for the Invoice Class

Program 11.26 File Invoice.h

```
1  /*****
2  * The interface file for the Invoice class *
3  *****/
4  #ifndef INVOICE_H
5  #define INVOICE_H
6  #include "product.h"
7
8  class Invoice
9  {
10     private:
11         int invoiceNumber;
12         double invoiceTotal;
13     public:
14         Invoice (int invoiceNumber);
15         ~Invoice ( );
16         void add (int quantity, Product product);
17         void print ( ) const;
18 };
19 #endif
```

Implementation File for Invoice Class

Program 11.27 File *invoice.cpp*

```
1  /*****
2   * The implementation file for Invoice class *
3   *****/
4  #include "invoice.h"
5
6  // Constructor
7  Invoice :: Invoice (int invNum)
8  : invoiceNumber (invNum), invoiceTotal (0.0)
9  {
10 }
11 // Destructor
12 Invoice :: ~Invoice ( )
13 {
14 }
15 // Add member function
16 void Invoice :: add (int quantity, Product product)
17 {
18     invoiceTotal += quantity * product.getPrice ();
19 }
20 // Print member function
```

A Comprehensive Example

Program 11.27 File *invoice.cpp*

```
21 void Invoice :: print ( ) const
22 {
23     cout << "Invoice Number: " << invoiceNumber << endl;
24     cout << "Invoice Total: " << invoiceTotal << endl;
25 }
```

Application File to Test the Invoice Class

Program 11.28 File *application.cpp*

```
1  /*****
2   * The application file to test the Invoice class          *
3   *****/
4  #include "invoice.h"
5
6  int main ( )
7  {
8      // Instantiation of two products
9      Product product1 ("Table", 150.00);
10     Product product2 ("Chair", 80.00);
11     // Creation of invoice for the two product
12     Invoice invoice (1001);
13     invoice.add (1, product1);
14     invoice.add (6, product2);
15     invoice.print ();
16     return 0;
17 }
```

Run:

Invoice Number: 1001

Invoice Total: 630

PROGRAMS DESIGN

We discuss two projects.

The first shows how to create a Tokenizer class, the second shows how to simulate the registration process at a university.

A Tokenizer Class Part 1

A common problem is to tokenize a string of characters using a list of delimiters.

For example, we may need to extract words from a text. The words in a text are separated by spaces and new-line characters.

The words in this case are called tokens and the characters that separate the words are delimiters. For example, the following string has seven tokens (words) in it.

"This is a book about C++ language"

A Tokenizer Class Part 2

The C++ language has no class whose objects can tokenize a string, but we can create a class to do so.

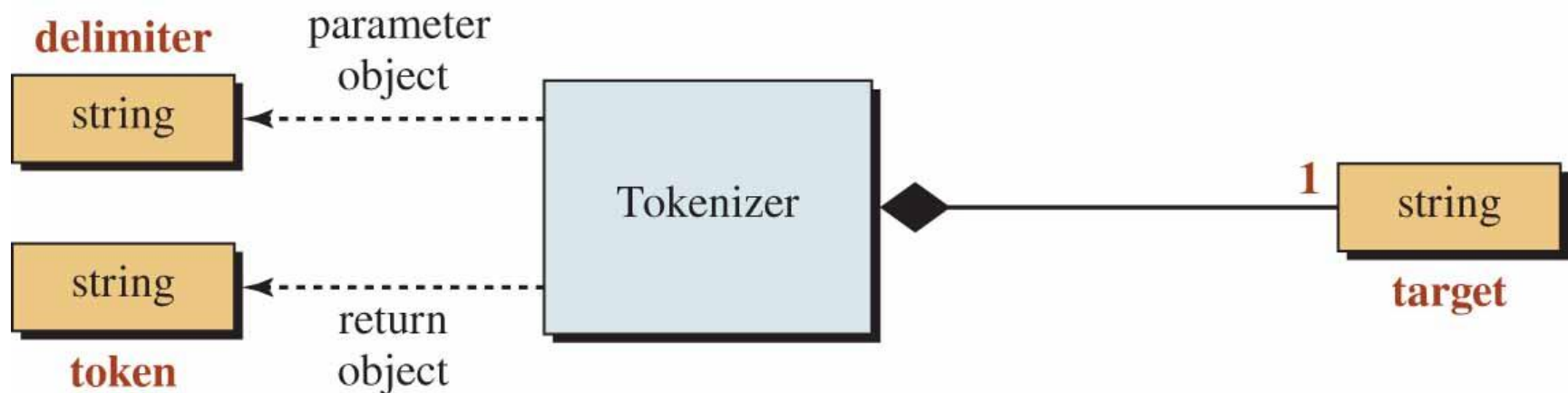
The string library class has member functions to search a string to find a character in a string or to find a character which is not in a string.

We can use these functions in the string library to create a Tokenizer class.

Relationship Between Classes Part 1

Figure 11.25 shows the UML class diagram that depicts the relationship between classes.

Figure 11.25 *Relationship between classes*



Relationship Between Classes Part 2

The Tokenizer class uses the string class three times.

It uses it to create the target string, the delimiter string, and the finally to create tokens.

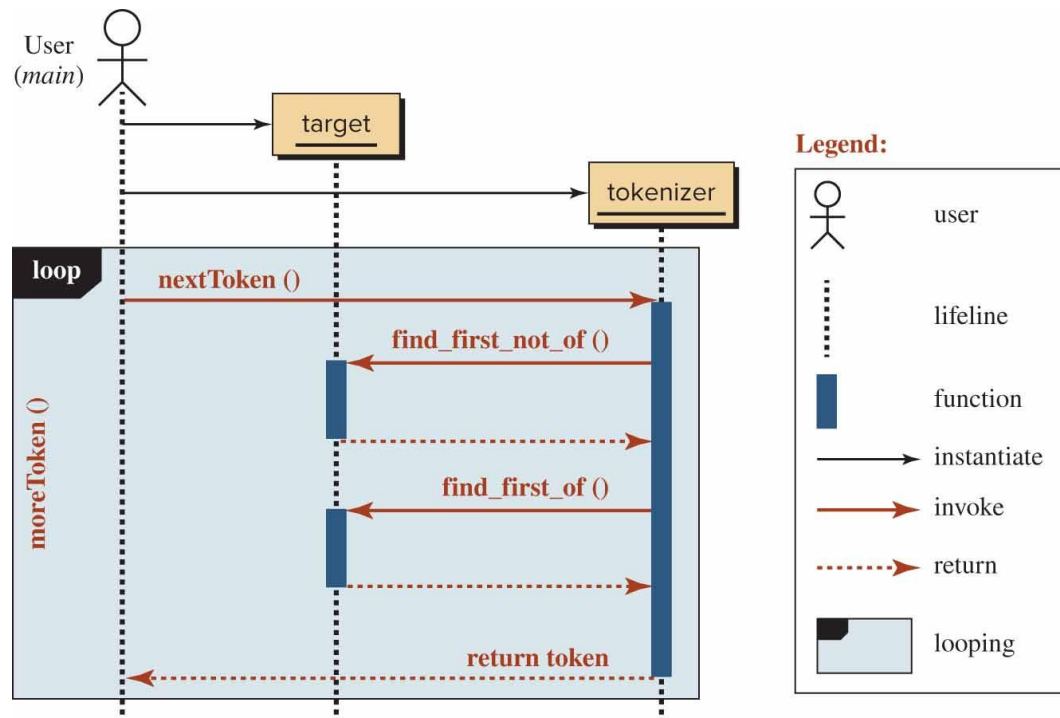
The relationship between the tokenizer object and the target object is composition.

The relationship between the tokenizer object and tokens is dependency.

Sequence Diagram

Figure 11.26 shows the interaction between objects in a sequence diagram.

Figure 11.26 *Sequence diagram for the Tokenizer design*



Tokenizer Class Programs

Since the string class (target) is a library class with predefined public interfaces, we have to create only one class, *Tokenizer*. Its interface file is shown in Program 11-29.

We also have one implementation file that implements the Tokenizer class (Program 11-30).

The user can create an application file to create and use objects of the Tokenizer class (Program 11-31).

Interface File for the Tokenizer Class Part 1

Program 11.29 File tokenizer.h

```
1  /*****
2  * The interface file for the Tokenizer class      *
3  *****/
4  #ifndef TOKENIZER_H
5  #define TOKENIZER_H
6  #include <iostream>
7  #include <string>
8  using namespace std;
9
10 class Tokenizer
11 {
12     private:
13         string target;
14         string delim;
15         int begin;
16         int end;
17     public:
18         Tokenizer (const string& target, const string& delim);
19         ~Tokenizer ();
20         bool moreToken() const;
```

Interface File for the Tokenizer Class Part 2

Program 11.29 *File tokenizer.h*

```
21         string nextToken() ;  
22     };  
23 #endif
```


Implementation File for the Tokenizer Class Part 1

Program 11.30 File tokenizer.cpp

```
1  /*****
2   * The implementation file for the Tokenizer class      *
3   *****/
4  #include "tokenizer.h"
5
6  // Constructor
7  Tokenizer :: Tokenizer (const string& tar, const string& del)
8  : target (tar), delim (del)
9  {
10     begin = target.find_first_not_of (delim, 0);
11     end = target.find_first_of (delim, begin);
12 }
13 // Destructor
14 Tokenizer :: ~Tokenizer()
15 {
16 }
17 // Checks for more tokens
18 bool Tokenizer :: moreToken ( ) const
19 {
20     return (begin != - 1);
```

Implementation File for the Tokenizer

Class Part 2

Program 11.30 *File tokenizer.cpp*

```
21  }
22  // Returns the next token
23  string Tokenizer :: nextToken ( )
24  {
25      string token = target.substr (begin, end - begin);
26      begin = target.find_first_not_of (delim, end);
27      end = target.find_first_of(delim, begin);
28      return token;
29  }
```

Application File to Test the Tokenizer Class

Part 1

Program 11.31 File *app.cpp*

```
1  /*****
2   * The application file to test the Tokenizer class      *
3   *****/
4  #include "tokenizer.h"
5
6  int main ( )
7  {
8      // The target string that needs to be tokenized
9      string target ("This is the string to be tokenized. \n");
10     // The delimit string defines the set of separators
11     string delimit (" \n"); // Delimiter made of ' ' and '\n'
12     // Instantiation of tokenizer object
13     Tokenizer tokenizer (target, delimit);
14     // Traversing the target string to find tokens
15     while (tokenizer.moreToken ())
16     {
17         cout << tokenizer.nextToken () << endl;
18     }
19     return 0;
20 }
```

Application File to Test the Tokenizer Class

Part 2

Program 11.31 *File app.cpp*

Run:

```
This  
is  
the  
string  
to  
be  
tokenize.
```

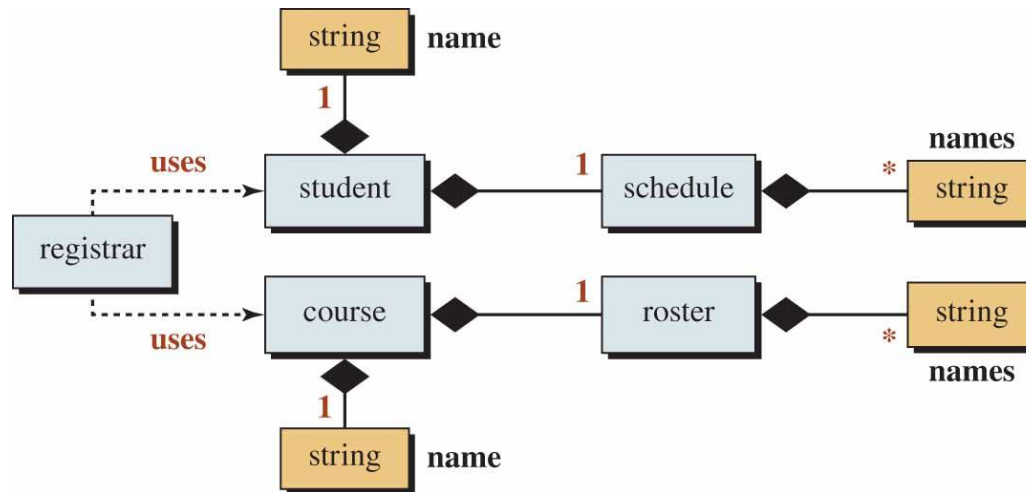
Registration

We use association and dependency between classes to design a simple registration system for a small department in a college or university.

UML Class Diagram.

We use six classes as shown in Figure 11.27.

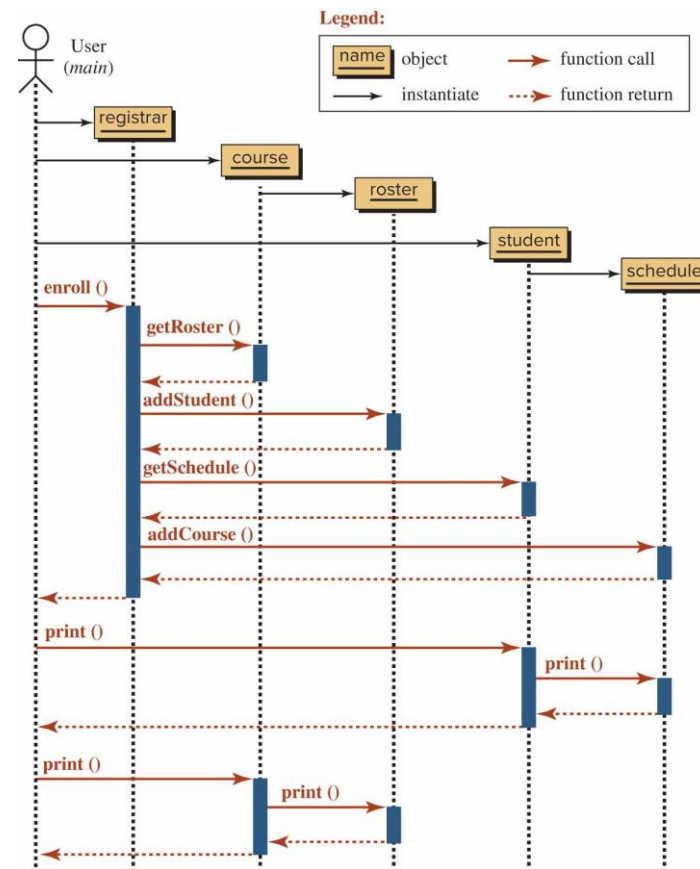
Figure 11.27 *Relationship between classes in registrar*



UML Sequence Diagram

Sequence diagram for the registrar project.

Figure 11.28 *Sequence diagrams for classes in registrar*



[Access the text alternative for slide images.](#)

Interface File for the Class CourseRoster

Part 1

Program 11.32 File courseRoster.h

```
1  /*****
2   * The interface file for the class CourseRoster      *
3   *****/
4  #ifndef COURSEROSTER_H
5  #define COURSEROSTER_H
6  #include <string>
7  #include <iostream>
8  #include <cassert>
9  using namespace std;
10
11 // Class Definition
12 class CourseRoster
13 {
14     private:
15         int size;
16         string* stdNames;
17     public:
18         CourseRoster ();
19         ~CourseRoster ();
20         void addStudent (string studentName);
```

Interface File for the Class CourseRoster

Part 2

Program 11.32 *File courseRoster.h*

```
21         void print () const;  
22     };  
23 #endif
```


Implementation File for Class CourseRoster Part 1

Program 11.33 File courseRoster.cpp

```
1  /*****
2   * The implementation file for class CourseRoster      *
3   *****/
4  #include "courseRoster.h"
5
6  // Constructor
7  CourseRoster :: CourseRoster ()
8  :size (0)
9  {
10     stdNames = new string [20];
11 }
12 // Destructor
13 CourseRoster :: ~CourseRoster ( )
14 {
15     delete [ ] stdNames;
16 }
17 // Definition of addStudent function
18 void CourseRoster :: addStudent (string studentName)
19 {
20     stdNames [size] = studentName;
```

Implementation File for Class CourseRoster Part 2

Program 11.33 File courseRoster.cpp

```
21     size++;
22 }
23 // Definition of print function
24 void CourseRoster :: print () const
25 {
26     cout << "List of Students" << endl;
27     for (int i = 0; i < size; i++)
28     {
29         cout << stdNames[i] << endl;
30     }
31     cout << endl;
32 }
```

Interface File for the Class Course Part 1

Program 11.34 File course.h

```
1  /*****
2  * The interface file for the class Course *
3  *****/
4  #ifndef COURSE_H
5  #define COURSE_H
6  #include <cassert>
7  #include <string>
8  #include <iostream>
9  #include "courseRoster.h"
10 using namespace std;
11
12 // Class Definition
13 class Course
14 {
15     private:
16         string name;
17         int units;
18         CourseRoster* roster;
19     public:
20         Course (string name, int units);
```

Interface File for the Class Course Part 2

Program 11.34 File course.h

```
21      ~Course ();  
22      string getName() const;  
23      CourseRoster* getRoster () const;  
24      void addStudent (string name);  
25      void print () const;  
26  };  
27 #endif
```

Implementation File for the Class Course

Part 1

Program 11.35 File *course.cpp*

```
1  /*****
2   * The implementation file for the class Course          *
3   *****/
4  #include "course.h"
5
6  // Constructor
7  Course :: Course (string nm, int ut)
8  : name (nm), units (ut)
9  {
10     roster = new CourseRoster;
11 }
12 // Destructor
13 Course :: ~Course ()
14 {
15 }
16 // Definition of getName function
17 string Course :: getName() const
18 {
19     return name;
20 }
```

Implementation File for the Class Course

Part 2

Program 11.35 File *course.cpp*

```
21 // Definition of addStudent function
22 void Course :: addStudent (string name)
23 {
24     roster ->addStudent (name);
25 }
26 // Definition of getRoster function
27 CourseRoster* Course :: getRoster () const
28 {
29     return roster;
30 }
31 // Definition of print function
32 void Course :: print () const
33 {
34     cout << "Course Name: " << name << endl;
35     cout << "Number of Units: " << units << endl;
36     roster -> print ();
37 }
```

Interface File for the Class StudentSchedule Part 1

Program 11.36 File studentSchedule.h

```
1  /*****
2  * The interface file for the class StudentSchedule      *
3  *****/
4  #ifndef STUDENTSCHEDULE_H
5  #define STUDENTSCHEDULE_H
6  #include <string>
7  #include <iostream>
8  #include <cassert>
9  using namespace std;
10
11 // Class Definition
12 class StudentSchedule
13 {
14     private:
15         int size;
16         string* courseNames;
17     public:
18         StudentSchedule ();
19         ~StudentSchedule ();
20         void addCourse (string course);
```

Interface File for the Class StudentSchedule Part 2

Program 11.36 *File studentSchedule.h*

```
21         void print () const;  
22     };  
23 #endif
```


Implementation File for the Class StudentSchedule 1

Program 11.37 File studentShedule.cpp

```
1  /*****
2   * The implementation file for the class StudentSchedule      *
3   *****/
4  #include "studentSchedule.h"
5
6  // Constructor
7  StudentSchedule :: StudentSchedule ()
8  :size (0)
9  {
10     courseNames = new string [5];
11 }
12 // Destructor
13 StudentSchedule :: ~StudentSchedule ()
14 {
15     delete [ ] courseNames;
16 }
17 // Definition of addCourse function
18 void StudentSchedule :: addCourse (string name)
19 {
20     courseNames [size] = name;
```

Implementation File for the Class StudentSchedule 2

Program 11.37 File studentShedule.cpp

```
21     size++;
22 }
23 // Definition of print function
24 void StudentSchedule :: print () const
25 {
26     cout << "List of Courses" << endl;
27     for (int i = 0; i < size; i++)
28     {
29         cout << courseNames[i] << endl;
30     }
31     cout << endl;
32 }
```

Interface File for the Class Student Part 1

Program 11.38 File student.h

```
1  /*****
2  * The interface file for the class Student          *
3  *****/
4  #ifndef STUDENT_H
5  #define STUDENT_H
6  #include <cassert>
7  #include <string>
8  #include <iostream>
9  #include "studentSchedule.h"
10 using namespace std;
11
12 // Class Definition
13 class Student
14 {
15     private:
16         string name;
17         StudentSchedule* schedule;
18     public:
19         Student (string name);
20         ~Student ();
```

Interface File for the Class Student Part 2

Program 11.38 File student.h

```
21     string getName () const;  
22     StudentSchedule* getSchedule () const;  
23     void addCourse (string name);  
24     void print () const;  
25 };  
26 #endif
```

Implementation File for the Class Student

Part 1

Program 11.39 File student.cpp

```
1  /*****
2   * The implementation file for the class Student          *
3   *****/
4  #include "student.h"
5
6  // Constructor
7  Student :: Student (string nm)
8  :name (nm)
9  {
10     schedule = new StudentSchedule;
11 }
12 // Destructor
13 Student :: ~Student ()
14 {
15 }
16 // Definition of getName function
17 string Student :: getName () const
18 {
19     return name;
20 }
```

Implementation File for the Class Student

Part 2

Program 11.39 File student.cpp

```
21 // Definition of getSchedule function
22 StudentSchedule* Student :: getSchedule () const
23 {
24     return schedule;
25 }
26 // Definition of addCourse function
27 void Student :: addCourse (string name)
28 {
29     schedule -> addCourse (name);
30 }
31 // Definition of print function
32 void Student :: print () const
33 {
34     cout << "Student name: " << name << endl;
35     schedule -> print ();
36 }
```

Interface File for the Class Registrar

Program 11.40 File registrar.h

```
1  /*****
2   * The interface file for the class Registrar *
3   *****/
4  #ifndef REGISTRAR_H
5  #define REGISTRAR_H
6  #include "course.h"
7  #include "student.h"
8
9  // Class Definition
10 class Registrar
11 {
12     public:
13         Registrar ();
14         ~Registrar();
15         void enroll (Student student, Course course);
16 };
17 #endif
```

Implementation File for the Class Registrar

Program 11.41 File registrar.cpp

```
1  /*****
2   * The implementation file for the class Registrar      *
3   *****/
4  #include "registrar.h"
5
6  // Constructor
7  Registrar :: Registrar ()
8  {
9  }
10 // Destructor
11 Registrar :: ~Registrar ()
12 {
13 }
14 // Enroll function
15 void Registrar :: enroll (Student student, Course course)
16 {
17     (course.getRoster ()) -> addStudent (student.getName());
18     (student.getSchedule()) -> addCourse (course.getName());
19 }
```


Application File Tests Classes in Registration Project 1

Program 11.42 File *application.cpp*

```
1  /*****
2  * The application file to test classes in Registration project *
3  *****/
4  #include "registrar.h"
5
6  int main ( )
7  {
8      // Instantiation of a Registrar object
9      Registrar registrar;
10     // Instantiation of three student objects
11     Student student1 ("John");
12     Student student2 ("Mary");
13     Student student3 ("Ann");
14     // Instantiation of three course objects
15     Course course1 ("CIS101", 4);
16     Course course2 ("CIS102", 3);
17     Course course3 ("CIS103", 3);
18     // Let the registrar object enroll students in the courses
19     registrar.enroll (student1, course1);
20     registrar.enroll (student1, course2);
```

Application File Test Classes in Registration Project 2

Program 11.42 File application.cpp

```
21 registrar.enroll (student2, course1);
22 registrar.enroll (student2, course3);
23 registrar.enroll (student3, course1);
24 // Printing the information about each student
25 student1.print();
26 student2.print();
27 student3.print();
28 // Printing information about each course
29 course1.print();
30 course2.print();
31 course3.print();
32 return 0;
33 }
```

Run:

Student name: John

List of Courses

CIS101

CIS102

Application File Tests Classes in Registration Project 3

Program 11.42 *File application.cpp*

```
Student name: Mary
List of Courses
CIS101
CIS103

Student name: Ann
List of Courses
CIS101

Course Name: CIS101
Number of Units: 4
List of Students
John
Mary
Ann
Course Name: CIS102
Number of Units: 3
List of Students
John
```

Application File Tests Classes in Registration Project 4

Program 11.42 *File application.cpp*

Course Name: CIS103

Number of Units: 3

List of Students

Mary